Applying formal verification to the `etcd` configuration service using concurrent separation logic

Paper #293

Abstract

Formal verification offers the potential of eliminating large classes of bugs in complex distributed systems. However, verification of widely-used distributed system implementations has remained out of reach owing to their size.

This paper brings the benefits of verification to the widely-deployed `etcd` distributed configuration service by verifying just a few specific functions instead of the entire system. The key is Concurrent Separation Logic (CSL), an existing modular verification approach which allows for reasoning about individual functions upholding global properties such as linearizability—*even without verifying the entire system.* Dingo, our new verification tool, brings this benefit of CSL reasoning to off-the-shelf Go code, and `etcd` in particular. The results of our `etcd` case studies demonstrate the value obtained by verifying just a few functions: we helped developers debug and fix a subtle linearizability bug with read-only Raft operations, found various bugs in `etcd`'s lease-based client-side caching library, and fixed a data race and correctness bug in `etcd`'s new watch-based cache.

1 Introduction

Formal verification offers the potential of eliminating large classes of bugs, and prior work has shown that it is possible to formally verify many kinds of systems, including OS kernels [9, 13, 22], security monitors [11, 26, 42], distributed key-value stores [15, 36], databases [7], storage systems [8, 14, 24], JITs [28, 37], and hardware security modules [2]. Researchers have introduced many techniques and proof patterns to make it possible to verify systems of this complexity.

However, formal verification of real-world deployed distributed systems has been limited. For example, AWS developers use TLA+ to reason about protocols, but that does not extend to the implementation [29]. Lightweight techniques have been applied to implementations, such as the S3 ShardStore [3], but these do not capture sophisticated distributed systems properties like linearizability and consistency. Many verification researchers, on the other hand, have focused on verifying implementations written specifically for verification. This paper explores whether and how modern verification techniques, that can capture properties like linearizability, can be applied to existing distributed system implementations.

Specifically, this paper reports on our experience applying formal verification techniques to `etcd`, an important widely-used and open-source distributed system, used by Kubernetes, among others. `etcd` is a configuration service, analogous to Chubby [4] and ZooKeeper [18], used by other systems to implement their own fault tolerance plans, such as by using `etcd` for leader election or storing the authoritative cluster state in `etcd`. `etcd` is implemented as a replicated state machine using Raft [31], provides a key-value store interface, and supports a wide range of features for coordination, including leases, sessions, and watches. `etcd` is implemented in about 60,000 lines of Go code, plus another 80,000 lines of tests. This meant `etcd` was too big for complete verification.

Instead, we focused on verifying individual functions using Concurrent Separation Logic (CSL) [21, 30], a modular verification technique. We built Dingo, a new verification tool based on the Grove CSL library [36]. Unlike Grove, Dingo supports verifying off-the-shelf Go code. Dingo includes formal semantics and CSL reasoning principles for a significant fragment of the Go programming language. We have publicly released Dingo as open-source software, so that others can use it for verifying their Go-based systems software. The full URL will be made available in a non-anonymous version.

Using Dingo, we targeted three components in `etcd`: the Raft implementation; an old, lease-based caching library; and a new, watch-based caching library. We chose these components because they deal with subtle concurrency and distributed systems issues, such as lease expiration; because they implement sophisticated optimizations, such as read-only operations in Raft; and because we (as well as `etcd`'s open-source developers) suspected they might have issues.

In the Raft implementation, we focused on a particularly tricky issue with an optimization for read-only operations that was puzzling the `etcd` developers. By formalizing the invariants associated with the Raft data structures for read-only operations, we helped the developers clarify the root cause of the bug they were seeing and implemented a more performant fix for the problem than what they were planning to implement. We further proved the correctness of the fixed implementation, ensuring that read-only operations achieve linearizability. The proof also allowed us to convince the `etcd` developers that certain checks (on slice bounds) were unnecessary. The developers adopted our proposed fix.

In the experimental lease-based client caching library, we suspected there might be linearizability bugs due to improper handling of lease expiration. We formulated invariants and started proving the `Get` function, which led us to discover several design issues related to concurrency that lead to linearizability violations. We also started proving `Put`, and discovered more linearizability violations due to improper handling of concurrent, out-of-order responses. While `etcd` developers

```
1  func (k *CacheKv) Get(key string) string {
2    k.mu.RLock()
3    cv, ok := k.cache[key]
4    if ok && cv.validLease() {
5      k.mu.RUnlock()
6      return cv.val
7    }
8    k.mu.RUnlock()
9    return DecodeValue(k.server.Get(key)).val
10 }
```

Figure 1: An example function from a caching key-value library, modeled after `etcd`'s leasing library.

confirmed some of the issues, they did not merge our fix, possibly because they had independently started working on an entirely new watch-based caching library.

Finally, we looked at the new caching library, which does not use leases. We chose a function that had concurrency and correctness bugs, and then implemented and verified a fix. The proof helped us avoid a bug in our initial attempted fix.

Across these case studies, CSL's modular verification techniques were crucial in enabling us to apply formal verification to `etcd`. CSL enabled us to encode global properties such as linearizability in a function-local way, and to verify arbitrary functions of interest in the middle of a large codebase. The key to this was using CSL's powerful ownership reasoning to write invariants and postulates for other parts of the codebase. This also helped us to develop bug fixes that have been adopted by `etcd`, even without verifying all of `etcd` or even a single complete module inside `etcd`.

The contributions of this paper are (1) verifying existing, widely-deployed distributed systems software using CSL; (2) Dingo, an open-source verification tool for Go code, based on CSL, that works for existing Go code that was not written with verification in mind; and (3) two findings from our experience applying verification to `etcd`: that even in large, widely-deployed distributed systems, CSL enables machine-checked proofs of correctness for individual tricky functions and that even without complete verification, CSL helps find and fix serious bugs.

2 Approach and background: CSL

Concurrent Separation Logic (CSL) [21, 30, 36] allows verifying one function at a time, even when this function is part of a larger system running concurrently with other threads and machines, and even when this function accesses shared state like memory, a global clock, or the network at the same time that other functions access it. This requires formally specifying all the ways the function might interact with the rest of the system.

As a running example, consider the function shown in Figure 1 from a caching key-value client library. The client `k` maintains a cache of key-value pairs, `k.cache`, which are supposed to be a subset of the key-value pairs at the server `k.server`. Each cached key-value pair `cv` is valid for some period of time, as checked by `cv.validLease()`. If some key is not present (or valid) in the cache, the client falls back

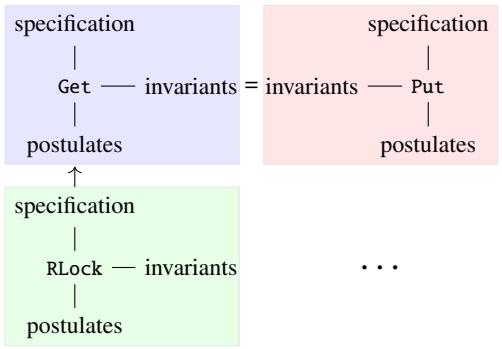


Figure 2: An overview of the main parts involved in CSL specification of the function `Get`: postulates for functions invoked by `Get` (e.g., `RLock` for a read-write mutex), a specification for `Get`, and invariants for any shared state accessed by `Get` and other functions (e.g., `Put`). Each shaded box represents a theorem for an individual function. Proofs about individual functions can be linked together by using the same invariants (e.g. for `Put`) or by having a specification match a postulate (e.g. for `RLock`).

to calling `k.server.Get()`. Multiple threads can invoke the client's `k.Get()` function concurrently; the implementation protects its state using a read-write mutex `k.mu`, which allows multiple concurrent readers. Other functions in this library are responsible for adding a cached key-value pair to `k.cache`, for implementing put operations, etc.

2.1 Individual function specifications

As illustrated in Figure 2, the developer can prove a theorem about this function by assuming postulates for functions invoked by `k.Get()` (namely, `cv.validLease()`, `k.server.Get()`, `k.mu.RLock()`, and `k.mu.RUnlock()`), defining invariants about shared state (namely, the `k` structure, including `k.cache`, `k.server`, and even `k.mu`), and establishing a specification for the function being verified, `k.Get()`. Invariants are logical propositions about shared state. Per-function specifications and postulates are written as a *Hoare triple* using pre- and post-conditions such as $\{P\}$ `k.Get()` $\{Q\}$, which denotes that callers can invoke `k.Get()` if they can establish precondition P and can assume the postcondition Q holds when the function returns [17]. This results in proofs that can reason about code line-by-line in terms of pre- and post-conditions. Note that this only covers safety properties; we do not reason about liveness.

CSL extends the logic used to specify pre- and post-conditions in a way that enables them to capture shared state and concurrency, while still preserving per-function reasoning. In particular, CSL provides *ownership* to encode a discipline about which threads are allowed to update or access different parts of state, *ghost state* to explicitly reason about the logical behavior of functions, and *ghost callbacks* to reason about concurrent operations without global reasoning.

2.2 Ownership

In `k.Get()`, ownership allows for reasoning about `k.cache` without directly reasoning about other threads accessing the same `k` object. The postulated postcondition of `k.mu.RLock()`

is the points-to proposition $\texttt{k.cache} \mapsto_q C$. This proposition not only asserts that the contents of `k.cache` is C, but also denotes *ownership* of `k.cache` (the q specifically denotes fractional, read-only ownership). Ownership of $\texttt{k.cache} \mapsto_q C$ means that other threads cannot own the logical resources they would need to modify `k.cache`. It allows the developer verifying `k.Get()` to reason about the lookup performed by `k.cache[key]` without worrying about other threads modifying `k.cache`, precisely because it is an assertion about what other threads can and cannot do (namely, it is an assertion that `k.cache` must remain equal to C).

This assertion can be thought of initially as an assumption about other threads in this module—that is, that they follow the locking rules. However, as part of verifying any function in CSL, the developer must prove that the code follows CSL's rules regarding ownership, and therefore ensure that the code maintains the concurrency assumptions encoded by ownership. In particular, if another function tried to modify `k.cache`, its CSL proof would require ownership of the $\texttt{k.cache} \mapsto C$ resource, which the logic forbids as long as the `k.Get` thread owns $\texttt{k.cache} \mapsto_q C$. Releasing the lock with `k.mu.RUnlock()` requires the `k.Get` thread's proof to return this resource. Once the lock is released, other threads can potentially acquire the mutex in writer mode, so returning the resource allows the proofs of these other threads to justify modifying `k.cache` by obtaining that resource when acquiring the write lock.

2.3 Ghost state

In `k.Get()`, CSL's ghost state allows specifying the relationship between the contents of `k.cache` and the state at the server `k.server`. The proof relies on the invariant that every key-value pair in `k.cache` either has the same value as in `k.server`, or its lease is expired. However, the proof of `k.Get()` cannot refer to the physical state of the server, or even to the complete logical state of the server's key-value map, since uncached keys could change due to concurrent operations from other machines.

To get around this, the proof can use ownership in combination with *ghost state*, which is auxiliary state introduced only for use in the proof and which can correspond to physical state in a proof-defined way. Specifically, the proof introduces a ghost state resource representing a specific key-value pair in the server, written as $\texttt{key} \mapsto^{\texttt{k.server}} \texttt{val}$. This resource corresponds to the logical value of the key, rather than the physical layout of the server's memory. The fact that the resource correctly captures the server's state is an assumption from the perspective of the `k.Get()` proof, but will be an obligation for the proof of the server's code, if or when the developer chooses to verify it.

In particular, when `k.mu.RLock()` returns, the proof of `k.Get()` receives both ownership of $\texttt{k.cache} \mapsto_q C$ and, for every key-value pair `cv` in C, ownership of the fact that either $\texttt{cv.key} \mapsto_q^{\texttt{k.server}} \texttt{cv.val}$ or that `cv`'s lease expired. This allows the proof of `k.Get()` to conclude that, if the lease is valid (in particular, if `cv.validLease()` returns true), then `cv.val` must be the same as the server's value for `key`.

$$
\begin{array}{l}
\forall\, s, \forall\, Q, \\
\left\{
\begin{array}{l}
\exists\, \texttt{op_linearize}, \\
\{\top\} \\
\quad \texttt{op_linearize()} \\
\left\{
\begin{array}{l}
\exists\, \texttt{val}, \exists\, \texttt{op_finish},\ \texttt{key} \mapsto^{s} \texttt{val} * \\
\quad \{\texttt{key} \mapsto^{s} \texttt{val}\}\ \texttt{op_finish()}\ \{Q(\texttt{val})\}
\end{array}
\right\}
\end{array}
\right\} \\
\quad s.\texttt{Get(key)} \\
\{\textbf{ret}\ v,\ Q(v)\}
\end{array}
$$

Figure 3: Simplified postulate for `k.server.Get()`. Here, $*$ is the separating conjunction from CSL, and should be read as the "and separately" operator.

2.4 Ghost callbacks

In `k.Get()`, to postulate a spec for `k.server.Get()`, CSL's ghost callbacks help specify linearizability with pre- and post-conditions. When the key is not present in the cache, `k.Get()` invokes `k.server.Get()` to fetch the key from the server. Since there's no valid lease, other clients may concurrently modify the server state by invoking other functions, such as `k.server.Put()` or `k.server.Delete()`. The server provides linearizability, but linearizability is traditionally described as a property of the server's entire execution trace [16]. The postulate for the server `Get()` function must somehow capture this while avoiding reasoning about all of the other clients and all of the other functions like `Put()` and `Delete()`.

This is possible by using CSL's *ghost callbacks*, which are callbacks invoked in the proof at the linearization point to update some ghost state. Ghost callbacks allow one to specify that `k.server.Get()` is linearizable by referring to its linearization point without explicitly considering other clients and other operations. The postulate exposes the linearization point of each operation by promising to invoke a ghost callback function at its linearization point, whose job is to atomically apply the operation to the ghost state. In this case, the ghost state is the $\texttt{key} \mapsto^{\texttt{k.server}} \texttt{val}$ resource, representing a specific key-value pair on the server. (This is the idea of *logical atomicity* [19]; ghost callbacks allow specifying more sophisticated consistency models too [12], but we focus on linearizability here for simplicity.)

Figure 3 shows a simplified version of a postulate for `k.server.Get()` using ghost callbacks. It exposes the linearization point of `k.server.Get` with the callback ghost function `op_linearize`. Ghost callbacks use the notation of Hoare triples, but they describe a proof step and do not refer to physical code. Specifically, the precondition of `s.Get` requires the caller to logically provide a ghost callback function `op_linearize` that meets a specification that returns ownership of $\texttt{key} \mapsto^{\texttt{s}} \texttt{val}$ for some value `val` and ensures there exists another callback `op_finish`. This second callback must

```
func CallOp(srv *KvServer) {
  ip := srv.Get("service")
  srv := rpc.Dial(ip)
  srv.Op()
}
```

Figure 4: Example function that invokes RPC operation `Op` on a server whose IP address comes from `k.Get()`.

also satisfy a specification: `op_finish` can assume that it gets ownership of `key` $\mapsto^{\mathsf{S}}$ `val` and must return some postcondition $Q(\texttt{val})$. Q is a proposition chosen by the caller and can contain resource ownership. The preconditions about ghost callbacks essentially require the caller to prove that Q holds after linearization. In turn, `Get`'s postcondition gives $Q(v)$, where v is the physically returned value v, as a witness that linearization occurred with value v.

In a proof of `k.server.Get()`, these callbacks would be invoked at the linearization point for this `Get` operation. Specifically, the proof will invoke `op_linearize` to determine what `key` $\mapsto^{\texttt{k.server}}$ `val` resource the caller might know about at the linearization point, and then the proof will invoke `op_finish` to return that resource back to the caller immediately afterwards, logically obtaining some opaque assertion Q about `val`. When `k.server.Get()` eventually returns, the postcondition says that the return value will satisfy this Q assertion, even though the state of the server may have changed. Not shown in the simplified postulate is the requirement that `op_linearize` and `op_finish` must be invoked as a pair.

On the other side, the caller of `k.server.Get()` can combine this specification with its own plan for concurrent operations to reason about the return value. For example, if the caller has exclusive ownership of `key` $\mapsto^{\texttt{k.server}}$ `val` (e.g., due to having obtained a lock protecting that key), it can keep its resource and conclude that the return value will be exactly `val` by choosing $Q(v) \stackrel{\text{def}}{=} (v = \texttt{val}) \wedge \texttt{key} \mapsto^{\texttt{k.server}} \texttt{val}$.

As another example, the application might use the key-value store to find the IP address of the current server responsible for some service; the key specifies the service name, and the value is the IP address. Figure 4 shows a simple example of such an application, using the key-value server `srv` directly instead of going through the `CacheKv`. The application's proof can state an assertion (specifically, an invariant):

$$\mathsf{ValidServiceKvInv} \stackrel{\text{def}}{=} \exists\, \texttt{ip}, \text{“}\texttt{service}\text{”} \mapsto^{\texttt{k.server}} \texttt{ip} * \mathsf{ValidServiceIP}(\texttt{ip})$$

The proof of `CallOp` can then supply an `op_linearize` that returns the “`service`” $\mapsto^{\texttt{k.server}}$ `ip` resource from its invariant, and an `op_finish` that puts the resource back in the invariant while establishing $Q \stackrel{\text{def}}{=} \mathsf{ValidServiceIP}$. (The ValidServiceIP statement is, in turn, captured by a postulate about `rpc.Dial(ip)`, whose postcondition is a postulate about `rpc.Dial(ip).Op()`).

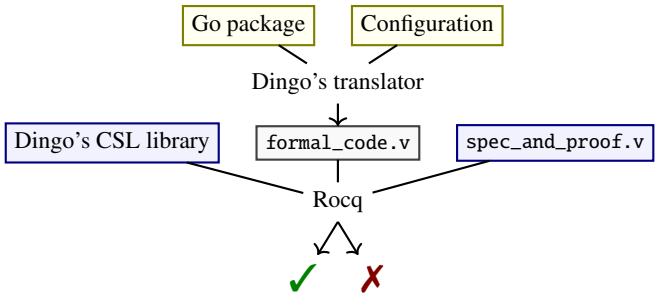


Figure 5: Dingo consists of a tool for turning Go source code into a formal definition, with a formal semantics and CSL library for Go. Users provide Go source code, a configuration for translation, and then can write specs and proofs using Dingo's CSL library written in the Rocq proof assistant.

2.5 Benefits of CSL

By following the CSL verification approach, one can reason about client code, such as `k.Get()` and `CallOp()`, independently of the server and other client operations. In particular, the linearizability specification for `k.server.Get()` did not have to mention other functions that it linearizes with, such as `k.server.Put()` or `k.server.Delete()`. Similarly, the specification and proof of `k.Get()` did not have to mention other methods that operate on `CacheKv`. If a developer chooses to apply verification to the server next, they will need to prove that the server indeed correctly implements these postulates and thereby demonstrate that the server's implementation provides linearizability for its operations. Similarly, if the developer chooses to verify other client code, they will need to prove that those functions are in agreement on the relevant postulates and invariants. For instance, if the developer verifies the code that calls `srv.Put("service", new_ip)`, they will have to prove that the invariant ValidServiceKvInv is maintained by showing ValidServiceIP(`new_ip`).

3 Dingo: verification of existing Go systems

To apply formal verification to `etcd`, we developed Dingo. Dingo consists of two important pieces, as shown in Figure 5: (1) a tool for giving formal semantics to Go source code by translating into a representation in the Rocq proof assistant; and (2) a Rocq library of CSL reasoning principles to enable verifying Go code. The former allows users to formalize real Go code by translating a subset of a Go package. The latter allows users to prove theorems about this formalized version of the original Go code.

Overall, Dingo consists of about 10k lines of Go for the translator tool, 4k lines of Rocq to define the semantics primitives, and 14k lines of Rocq for the untrusted library of CSL reasoning principles. These line counts include our new additions as well as some existing code we used from the prior Goose and Perennial frameworks.

3.1 Semantics for Go programs

Formulating theorems about Go code requires a formal definition for how Go programs execute, i.e., a semantics. Dingo

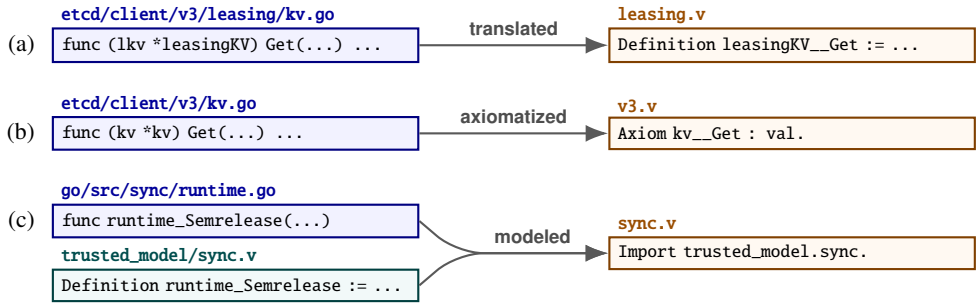


Figure 6: The different ways Dingo can be configured to turn Go source code into formal Rocq code.

supports a large fragment of Go (e.g., supporting goroutines, channels, interfaces, and generics). Dingo also supports three different translation modes that developers can choose from for each Go declaration, as shown in Figure 6. Each function or method has its own configuration.

For code that they want to verify, developers can configure a Go declaration to be *translated*. These declarations are given a complete formal definition by translation into a formal language in Rocq. The translation is done by a trusted component of Dingo. For instance, in Figure 6(a) the etcd function `leasingKV.Get` is translated into `leasingKV__Get`.

For code they do not want to verify, developers can configure a Go declaration to be *axiomatized*. This results in a placeholder formal definition with no body. For instance, in Figure 6(b), the etcd functions used by `leasingKV.Get` are axiomatized. This avoids needing to translate the vast majority of etcd.

For code to which they want to assign a manually written formal semantics, developers can configure a Go declaration to have a trusted *model*. This results in Dingo emitting no formal definition for the declaration. Instead, the developer writes a definition in Rocq that represents a simplified model of the code. Other translated code then refers to this model definition. For instance, in Figure 6(c), semaphores for RWMutex are marked as modeled, so the RWMutex translation actually refers to the hand-written semantics from `trusted_model/sync.v`. In contrast to axiomatizing code, this allows for proving a spec, albeit on a simplified model rather than the real implementation. This helps avoid unsound postulates and allows the developer to build confidence about a possible future specification for the code they are modeling.

3.2 CSL reasoning for Go primitives

To help reason about the formalized Go code, Dingo provides a library of CSL reasoning principles for a large subset of Go. These foundational reasoning principles and developers' higher-level proofs are written in the Iris CSL framework. From Iris, Dingo inherits the ability to use ownership, ghost state and ghost callbacks in pre and post-conditions.

For instance, in Go, map inserts and lookups are non-atomic. Dingo provides pre/postcondition specs for these map operations (these specs are verified against a trusted model for maps). Because these operations are not atomic, the specs must disallow concurrent modification and access. The specs do this by introducing a Go map points-to resource, written $m \overset{\text{map}}{\mapsto}_q M$. Here, m is the Go map value (which is actually a pointer to the underlying map data), $M : K \overset{\text{fin}}{\rightharpoonup} V$ is a mathematical mapping representing the abstract state of the map, and q is a positive fraction that's at most 1, allowing for splitting ownership into multiple fractions. The specs for insert and lookup are

$$\left\{m \overset{\text{map}}{\mapsto}_1 M\right\}\, m[k] = v \,\left\{m \overset{\text{map}}{\mapsto}_1 (M[k \leftarrow v])\right\}$$
$$\left\{m \overset{\text{map}}{\mapsto}_q M \wedge M(m) = v\right\}\, m[k] \,\left\{\textbf{ret}\ v,\ m \overset{\text{map}}{\mapsto}_q M\right\}.$$

The spec for insert requires complete ownership of the map—meaning no other parts of the proof can simultaneously own any positive fraction, thus making it impossible to prove a concurrent access safe. On the other hand, map lookup only requires some positive fraction q, which allows for proving concurrent threads looking up from a map at the same time. Dingo provides similar CSL reasoning principles for Go slices, structs, interfaces, and all the other Go features it supports. Dingo is complete enough to handle substantial existing Go code, including large parts of Go's standard library and `etcd`, as shown in the case studies described next.

4 Case studies

We applied CSL-style formal verification to several parts of `etcd`; this section describes our experience in doing so.

4.1 Read-only optimization in Raft

We selected an unresolved `etcd` bug report involving a linearizability violation of `etcd`'s key-value put and get operations. We verified a few functions that seemed suspicious based on the discussion in the bug report. Formalizing the invariants used by these functions helped us clarify the root cause of the bug (which was introduced in 2016) and propose a fix that was merged by the `etcd` maintainers.

The original bug report concerned a violation of linearizability in `etcd`'s robustness tests.[1] In particular, some read

[1]The URLs for our GitHub discussions and bug reports will be made available in a non-anonymous version.

requests were returning stale data. The etcd maintainers added additional debug logging and reran the failing test, which helped identify that the issue was related to the raft library's optimization for read-only requests.

The read optimization relates to Raft's term numbers and log of operations [31]. Raft is a majority-based consensus protocol for replicating a log of operations. Executing an operation requires committing it to the Raft log. Raft divides its execution into monotonically increasing terms, each having at most one leader; at any given moment, only the leader for the largest term number has an up-to-date log. Raft servers can serve linearizable read operations without committing them to the log by instead determining a safe log index, i.e., a log index that contains all previously committed operations. If a leader confirms that no newer leader has been elected, then its commitIndex is guaranteed to be a safe log index. A leader can confirm that no new leaders have started by sending a round of heartbeats to all Raft servers. Servers only respond to these heartbeats if they are still in the same term. Thus, if the leader receives a majority of heartbeat responses, then previously-started reads can safely be handled using the leader's commitIndex.

The original code. etcd's raft library's original implementation of this read-only optimization is shown in Figure 7. The Raft library uses the readOnly.readIndexQueue list to keep track of read-only requests waiting to execute on the leader. The addRequest function is called when a read-only request arrives, whereby the ID of that request (called its *context* and stored in m.Entries[0].Data for a read request message) is appended to the readIndexQueue. At a higher level, the leader sends out a heartbeat request to all other servers, with a heartbeat ID set to the ID of the read-only request itself. When the heartbeat responses come back, the recvAck function updates the ro.pendingReadIndex[context].acks map. Finally, when the acks map gets a majority of responses, the advance function is called. It finds the specified ID (m.Context) in ro.readIndexQueue, and returns the set of read-only requests leading up to that ID (so that the caller of this function can finish executing this entire batch of read-only requests).

The bug. The problem arises ultimately due to a combination of two facts: first, the Raft library uses the read request ID as the ID of the heartbeat request, and second, the etcd key-value store that uses this Raft library retries read requests with the same request ID. As a result, there could be multiple heartbeat requests with the same ID, and there's no way for the leader to tell them apart.

Specifically, the bug happens if there was a read-only request handled by a leader in the past, but the client never received the response. In the meantime, leader election happens, and a new leader is chosen. Now, the stale leader receives several new read requests from clients that haven't heard about the leader change yet, followed by a retransmitted read request from the first client. At this point, the stale leader sends out a heartbeat request with the ID of the retransmitted read. Now, the stale leader receives old heartbeat responses for its old heartbeat request using that ID (e.g., because they were delayed). The stale leader concludes it has a quorum (even though that is incorrect, since the quorum is actually for

```go
type readOnly struct {
  option           ReadOnlyOption
  pendingReadIndex map[string]*readIndexStatus
  readIndexQueue   []string
}

// Simplified version of Heartbeat message struct
type Message struct {
  Type    MessageType
  To      uint64
  From  uint64
  Context []byte
}

func (ro *readOnly) advance(m pb.Message) []*readIndexStatus {
  var (
    i     int
    found bool
  )

  ctx := string(m.Context)
  var rss []*readIndexStatus

  for _, okctx := range ro.readIndexQueue {
    i++
    rs, ok := ro.pendingReadIndex[okctx]
    if !ok {
      panic("cannot find corresponding read state from pending map")
    }
    rss = append(rss, rs)
    if okctx == ctx {
      found = true
      break
    }
  }

  if found {
    ro.readIndexQueue = ro.readIndexQueue[i:]
    for _, rs := range rss {
      delete(ro.pendingReadIndex, string(rs.req.Entries[0].Data))
    }
    return rss
  }

  return nil
}

func (ro *readOnly) addRequest(index uint64, m pb.Message) {
       s := string(m.Entries[0].Data)
       if _, ok := ro.pendingReadIndex[s]; ok {
              return
       }
       ro.pendingReadIndex[s] = &readIndexStatus{index: index, req: m, acks: make(map[uint64]bool)}
       ro.readIndexQueue = append(ro.readIndexQueue, s)
}

func (ro *readOnly) recvAck(id uint64, context []byte) map[uint64]bool {
       rs, ok := ro.pendingReadIndex[string(context)]
       if !ok {
              return nil
       }

       rs.acks[id] = true
       return rs.acks
}
```

Figure 7: Original (buggy) implementation of read-only optimization.

$\mathsf{IsReadCtx}(\texttt{readCtx}, Q) \stackrel{\text{def}}{=} \texttt{readCtx} \mapsto_{\square} Q$

$\mathsf{IsReadRequest}(Q) \stackrel{\text{def}}{=}$
$\exists\, \texttt{op_linearize},$
$\{\top\}$
$\texttt{op_linearize()}$
$\left\{\begin{array}{l}\exists\, \ell, \exists\, \texttt{op_finish},\ \mathsf{log} \mapsto \ell\ * \\ \quad \{\mathsf{log} \mapsto \ell\}\ \texttt{op_finish()}\ \{Q(\texttt{val})\}\end{array}\right\}$

$\mathsf{IsSafeRead}(\texttt{readIndex}, Q) \stackrel{\text{def}}{=}$
$\forall\, \ell, \mathsf{log} \sqsupseteq \ell \wedge \texttt{readIndex} \leq |\ell| \to Q(\ell).$

Figure 8: Definitions for propositions used in `addRequest` and `advance` specs.

a different heartbeat request). Now the stale leader executes a batch of read-only operations, up to the retransmitted read. However, since the leader was stale, it does not have the latest writes, and therefore the read requests (and ultimately the corresponding clients) observe stale data.

Trying to verify existing code. Since we (and the `etcd` developers) originally didn't understand the root cause of the bug as described above, we started our verification attempt by trying to verify the three functions that seemed relevant to the bug—namely, `advance`, `recvAck`, and `addRequest`. We formulated some initial specifications for the three functions: most importantly, `addRequest` requires a valid request and `advance` promises that the returned reads are safe to run.

The formalization of these specifications uses the definitions shown in Figure 8. In particular, the proposition $\mathsf{IsReadCtx}(\texttt{readCtx}, Q)$ denotes that the read request identified by `readCtx` has begun and is expecting postcondition Q. $\mathsf{IsReadRequest}$ defines what it means to be a valid request. Similar to Figure 3, its definition uses ghost callbacks to encode the fact that read operations with postcondition Q can be linearized. $\mathsf{IsSafeRead}$ denotes that `readIndex` is a safe read index for a read expecting postcondition Q. Using these propositions, the precondition of `addRequest` is effectively $\exists Q, \mathsf{IsReadCtx}(\texttt{m.Context}, Q) * \mathsf{IsReadRequest}(Q)$ and the postcondition of `advance` is $\exists Q, \mathsf{IsReadCtx}(\texttt{ctx}, Q) * \mathsf{IsSafeRead}(\texttt{idx}, Q)$ for every returned index `idx` and its corresponding context `ctx`. While these specifications appear complicated, they follow common patterns from other linearizability specifications, such as those from Grove [36].

Proving these specifications required us to reason about heartbeats, and more specifically, what guarantees the proof can expect from a heartbeat response. We proceeded to formulate invariants associated with heartbeats, along with lemmas that would hopefully help us take advantage of these invariants in our code proofs.

Invariants for heartbeats. To reason about how heartbeats help establish $\mathsf{IsSafeRead}$, the proof requires invariants associated with heartbeat requests and responses. The invariant associated with a message is the CSL proposition that the proof of a server gains when it receives that message. Because messages can be delayed and resent, the property captured by the CSL proposition must be *persistent*—the property must remain true forever once it is established.

At first glance, this appears to be at odds with the transient guarantees provided by heartbeats. When a server sends a heartbeat response, it confirms to the leader that it is not in a higher term. However, this is only true *at the time that the request was processed*. By the time a heartbeat response is received by the leader, the server that sent it may have received other messages and entered a new term.

By making use of ghost state, we were able to formulate persistent propositions in spite of this apparent transience. In the implementation, every round of heartbeats is tagged with a unique identifier that we call the `HbCtx` (to help match up the responses with the requests). Our invariant uses map ghost state that associates `HbCtx` to a set S of servers through a read-only, persistent points-to $\texttt{HbCtx} \mapsto_{\square} S$. The set S is picked in the proof to be the set of servers that was in a higher term than the leader at the moment that `HbCtx` was first used.

The invariant associated with a heartbeat request is

$$\mathsf{IsHeartbeatRequest}(\texttt{req}) \stackrel{\text{def}}{=}$$
$$\exists\, S,\ \texttt{req.HbCtx} \mapsto_{\square} S\ *\ (\forall s \in S, s.\texttt{Term} > \texttt{req.Term})$$

This asserts that some S has been selected for `HbCtx` such that every server in S is in a newer term than the request's term. The proposition is persistent because the value S associated with `HbCtx` cannot be changed and because server term numbers are monotonic. In fact, the notation $s.\texttt{Term} > \texttt{req.Term}$ actually refers to a piece of monotonic ghost state used to track server term numbers in the proof.

The invariant associated with a heartbeat response is

$$\mathsf{IsHeartbeatResponse}(\texttt{resp}) \stackrel{\text{def}}{=}$$
$$\exists\, S,\ \texttt{resp.HbCtx} \mapsto_{\square} S\ *\ \texttt{resp.From} \notin S.$$

This asserts that there is some set S associated with `HbCtx`, and the sender of the response is not in that set. This proposition is persistent. Since S is the set of higher-term servers at the moment that the `HbCtx` was first used, this persistent fact encodes the desired guarantee that `resp.From` was still in the same term as the leader at the time `HbCtx` was processed.

Heartbeat lemma proofs. After defining some high-level invariants about Raft (e.g., ensuring that if there is a leader in some term, then there is a quorum of servers in that term), we proved the following lemma about ghost state:

(GHOSTREAD)
$$\mathsf{IsRaftInv} * \mathit{term} \mapsto_{\text{commitIndex}} c * \mathsf{IsReadRequest}(Q) \implies$$
$$\mathsf{IsSafeRead}(c, Q) \vee \left(\begin{array}{l}\exists\, S,\ \mathsf{IsQuorum}(S) \wedge \\ \quad \forall s \in S, s.\texttt{Term} > \mathit{term}\end{array}\right)$$

This ensures that either a leader's `commitIndex` is safe for already-started reads, or else there is a quorum of servers in a new term.

Each heartbeat message corresponds to some read request. Using GHOSTREAD, the proof concludes that one of the two disjuncts holds. In the case that there is a quorum S of newer servers, we update the heartbeat ghost state to establish $\texttt{HbCtx} \mapsto_\square S$. This requires that there be no set already associated with `HbCtx`, i.e. it requires exclusive ownership of the unused heartbeat context. Since every server in S is in a higher term, the proof can establish $\mathsf{IsHeartbeatRequest}(\mathsf{req})$ for this `HbCtx`.

Suppose the code eventually receives a quorum T of heartbeat responses for `HbCtx`. Each response message satisfies $\mathsf{IsHeartbeatResponse}$. This implies that $\forall\ s \in T,\ s \notin S$. This implies that T and S are both quorums, but have no intersection. This is impossible, and thus rules out the right disjunct in the case that the code receives enough heartbeats.

Connecting heartbeat invariants to the code. Once we formalized what heartbeats can guarantee, we proceeded to connect the heartbeat invariants to the code. Specifically, the implementation must maintain two key properties. First, if an entry in `readIndexQueue` is confirmed, then all the entries before it in the queue must also be safe—this is needed because `advance` will return all of the preceding read operations as a single batch. Second, if a quorum of heartbeat responses for `readCtx` is received by the leader, then there was no new leader at the time that the heartbeats were first sent. At this point, the `etcd` developers posted an example execution trace that seemed to reproduce the bug, but didn't understand the root cause yet. Armed with the understanding of the invariants that have to hold, we were able to explain the root cause and produce a small test case. Namely, the first of these properties did not hold in the buggy implementation.

Fixing the bug. The `etcd` developers were originally considering several options to avoid the undesirable execution traces, including not batching reads (which would hurt read performance), or changing `etcd` to choose a new request ID when retransmitting a read request. Our proposal was to instead use an incrementing integer as the heartbeat request ID, which fixes the problem in a way that is independent of `etcd` (or the many other uses of `etcd`'s popular Raft library), and allows the Raft library to keep batching read requests.

Verifying the fixed code. With the above fix, we were able to establish the invariants needed for the code's correctness. In particular, the heartbeat invariant's set of servers $S_{\texttt{ctx}}$ represents the servers that have entered a new term at the time the heartbeat request `ctx` was created and sent out. When a leader receives a `HeartbeatResponse{ctx}` from server `s`, the heartbeat invariant guarantees that $\texttt{s} \notin S_{\texttt{ctx}}$. When a read request is passed to `addRequest`, it chooses the read index `idx` equal to the current committed index on that leader. As proved in GHOSTREAD, Raft's invariants guarantee that either the leader is active, and therefore fully knows the log of committed operations, or the leader is stale, and there exists some set S of servers in a new term such that $\mathsf{IsQuorum}(S)$. In case the leader is active, full knowledge of the log implies $\mathsf{IsSafeRead}(\texttt{idx}, Q)$. The proof combines these two cases by associating the following invariant for the ith read operation:

$$\mathsf{IsQuorum}(S_i) \vee \mathsf{IsSafeRead}(\texttt{idx}, Q) \tag{1}$$

To prove the correctness of batch-executing a group of read operations when the leader gets a quorum of responses to some heartbeat request, we add an invariant that

$$\forall i, \left(\bigcup_{1 \le j < i} S_j \right) \subseteq S_i$$

where S_i is the set of servers associated with the heartbeat request ID i (i.e., $\texttt{i} \mapsto_\square S_i$). The proof takes advantage of this invariant when proving that it is safe to execute the read operations in a batch. Specifically, when the leader gets a quorum of heartbeat responses for read request i, there cannot be also a quorum of servers in S_i (that is, $\neg\mathsf{IsQuorum}(S_i)$), due to the fact that there cannot be two disjoint quorums. Furthermore, by the above invariant, the proof deduces that $\neg\mathsf{IsQuorum}(S_j)$ for $j \le i$. The proof can then combine these facts with the invariant from Equation 1 for all of the read operations up to i to conclude that, since $\mathsf{IsQuorum}(...)$ is false, then it must be that $\mathsf{IsSafeRead}(...)$ holds, and therefore it is safe to execute each of them at their respective read index.

Summary. This case study of `etcd`'s Raft read-only optimization shows how verification helped deal with a particularly subtle bug that existed in the code for about 10 years; even the open bug report stayed unresolved for many months. Using formal verification, we were able to find the root cause of the linearizability violation, which enabled us to produce a test case for the `etcd` developers convincing them that this was the correct root cause. Formal verification also helped us design and implement a better fix with confidence. The `etcd` developers were originally cautious about our non-trivial change, and were considering going with their own, more conservative fixes, even at the cost of performance. However, laying out a sketch of the proof argument helped convince them the fix was right. They accepted our fix and it is now part of `etcd`'s Raft library.

We got all of this value from formal verification by verifying just 3 functions in `etcd`'s Raft library. Despite the fact that this bug is tightly coupled to the Raft protocol, we did not have to write a full specification for the Raft protocol itself. Instead, we wrote invariants for just the parts of the system that the 3 verified functions interacted with. In total, we wrote 958 lines of proof to cover 68 lines of code.

Our proof of the 3 functions ensures that their implementations meet the specifications we stated for them, under the

```go
// Package leasing serves linearizable reads from a local cache by acquiring
// exclusive write access to keys through a client-side leasing protocol.
// ...

func (lkv *leasingKV) Get(ctx context.Context, key string, opts ...v3.OpOption) (*v3.GetResponse, error) {
  return lkv.get(ctx, v3.OpGet(key, opts...))
}

func (lkv *leasingKV) get(ctx context.Context, op v3.Op) (*v3.GetResponse, error) {
  do := func() (*v3.GetResponse, error) {
    r, err := lkv.kv.Do(ctx, op)
    return r.Get(), err
  }
  if !lkv.readySession() {
    return do()
  }

  if resp, ok := lkv.leases.Get(ctx, op); resp != nil {
    return resp, nil
  } else {
    // ...
  }
}

func (lkv *leasingKV) readySession() bool {
  lkv.leases.mu.RLock()
  defer lkv.leases.mu.RUnlock()
  if lkv.session == nil {
    return false
  }
  select {
  case <-lkv.session.Done():
  default:
    return true
  }
  return false
}
```

Figure 9: Part of the code for the `leasing` library from `etcd`.

assumption that all of the stated postulates and invariants are maintained by other code in the system. Of course, the postulates and invariants could be wrong or incomplete (e.g., other bugs remaining in `etcd`), but our case study demonstrates that proving such theorems is nonetheless valuable.

This case study also highlights the importance of verifying the real code of a distributed system. Raft has been verified at the protocol level, and `etcd` has a TLA+ model of `raft`, but this bug was caused by a combination of subtle details, including how the implementation chooses heartbeat IDs, how the `etcd` application uses this Raft library (and re-uses request IDs for retransmissions), etc.

4.2 Leasing client-side cache

As a different type of case study in applying verification to `etcd`, we picked the `leasing` library that implements a client-side cache of `etcd`'s key-value store. We did not have specific bug reports, unlike in the read-only optimization case-study. Rather, one reason we chose it is that it is an experimental feature of `etcd`, so it has not been extensively tested and could benefit from verification to provide strong assurance. Another reason is that it is tricky: it involves both clients and servers, requires reasoning about concurrency and session expiration, and builds on top of large existing interfaces including `etcd`'s key-value store, `etcd`'s session library, etc.

```go
// Done returns a channel that closes when the lease is orphaned, expires, or
// is otherwise no longer being refreshed.
func (s *Session) Done() <-chan struct{} { return s.donec }
```

Figure 10: Code and comment for the `etcd` session library's `Done` method.

$$\forall session,$$
$$\{\texttt{is_session}(session)\}$$
$$\quad session.\texttt{Done()}$$
$$\{\textbf{ret}\ ch,\ \texttt{closed}(ch) \rightarrow \mathsf{Expired}(session)\}$$

Figure 11: Postulate for the session library `Done` method from Figure 10.

Figure 9 shows a fragment of code for `etcd`'s leasing library, focusing on the functions involved in handling a `Get` operation from the cache. The `lkv.get()` function first checks if the session is still valid by calling `lkv.readySession()`. If the session hasn't been set up yet, or has expired, `readySession()` returns `false`, and `lkv.get()` falls back to issuing a `Get` request to the server `lkv.kv` by calling the `do()` function closure. The implementation of `readySession()` obtains a read-lock that protects `lkv.session` and checks if the session is expired by checking whether the `lkv.session.Done()` channel is closed (by attempting to read from it, which is a common Go idiom). The session library closes this channel when it determines the session is expired or otherwise no longer valid.

We chose to start the verification of the leasing library with the `get()` function, because proving `get()` requires showing that the contents of the cache are consistent with the server state. If we were able to prove this, we would in effect capture the core of the correctness argument for the leasing library.

Specification. We started by writing a specification for `Get()`. While `etcd` does not come with formal specifications, the leasing library has an informal comment stating that its goal is to achieve linearizability. We used ghost callbacks, as described in §2, to formalize the fact that `Get()` should be linearizable. The ghost operation way of specifying linearizability allowed us to state that `Get()` is linearizable without having to mention any of the other operations that `Get()` might be linearized with respect to.

Postulates. Once we had a specification for `Get()`, we started proving it, going through its implementation line-by-line. As we considered each line in turn, we had to state postulates for calls to any functions used by `Get()`. In our proof, we decided to simply unfold (inline) the definitions of `get` and `readySession`, and thus the first function we called was `lkv.leases.mu.RLock()`. We stated a postulate for it, similar to the description in §2.2. (We later verified the `RWMutex` implementation, though we do not discuss its proof here.)

Next, we encountered a call to `lkv.session.Done()`. Conveniently, the session library provided an informal comment describing the intended behavior of this function, as shown in Figure 10. We captured this in a postulate for `lkv.session.Done()` as shown in Figure 11.

Invariant. Next, `get()` calls `lkv.leases.Get()`, which returns the locally-cached response, if any. The postulate for `lkv.leases.Get()` is fairly straightforward: it introduces a map ghost state representing the set of keys present in `lkv.leases`, and the postcondition says that the return value corresponds to that key's entry in the map. The interesting part, however, is the relation between this ghost map and the state of the server. We decided that it is likely to be the following invariant:

$$\begin{aligned}\mathsf{LeasingInv} \stackrel{\text{def}}{=} \mathsf{Expired}(\texttt{lkv.session}) \lor \\ \big(\forall\, \texttt{key}, \texttt{key} \in \texttt{lkv.leases} \to \\ \texttt{key} \mapsto^{\texttt{lkv.kv}} \texttt{lkv.leases[key]}\big)\end{aligned}$$

which states that either the session is expired, or every cached value in `lkv.leases` matches the current value of that key in the server's state. This invariant helps us capture assumptions about other threads on this client (namely, that they will not insert invalid entries into `lkv.leases`) as well as about the server and other clients (namely, that they will not be able to modify the server state in a way that violates this invariant), and proceed with verifying `lkv.get()`.

Proof. Finally, our proof has reached the `return resp, nil` statement in `lkv.get()`. At this point, we must prove that the return value meets the postcondition that we stated in our specification—namely, that it is a linearizable read against the server's logical state. To prove this, we can use the LeasingInv invariant stated above. The invariant has two possible cases, so our proof has to consider both of them. In the second case, the value in `lkv.leases` must correspond to a $\texttt{key} \mapsto^{\texttt{lkv.kv}}$ `lkv.leases[key]` resource, which we can use to demonstrate that returning `lkv.leases[key]` achieves linearizability. The first case, however, corresponds to an expired session where we have no correspondence between `lkv.leases` and the server state.

To prove that this case cannot happen, we need to prove that if this return statement is reached, then $\mathsf{Expired}(\texttt{lkv.session}) \to \mathsf{False}$. We know that the `lkv.session.Done()` channel was not closed, however, that is not enough to make use of the postcondition from the postulate in Figure 11. Namely, the session library guarantees that, if the channel is closed, the session is definitely expired, but not the other way around: if the session is expired, the channel might or might not be closed.

This turns out to be a bug with the leasing library that can lead to a non-linearizable execution of `lkv.Get()`. Namely, when a client's session expires, a background thread is responsible for closing the `lkv.session.Done()` channel. However, this thread might get delayed. In the meantime, another client could execute `Put()` and `Get()` operations on that key freely, since there is no active session. After those `Put()` and `Get()` calls complete, the first client executes `Get()`, which observes that the channel hasn't been closed yet (because that thread is still delayed), and returns the now-stale value from its cache, violating linearizability. We confirmed this by writing an explicit Go test case that demonstrates this incorrect execution, and filed a bug report with `etcd` with a proposed fix (introducing a new API call in the session library to actively check for validity). One of the `etcd` developers confirmed the issue, and agreed with our proposed approach, but hasn't merged our fix, possibly because `etcd` developers started working on an entirely new version of a client caching library to replace the `leasing` experiment (in parallel with us doing our verification of `leasing`).

Verifying Put. We also applied verification to the `Put()` function in the leasing library, and discovered another issue when an application issues concurrent `Put()` and `Delete()` operations through the leasing library. In this case, the responses to the server put and delete calls might come back to the client library out-of-order with respect to the order in which they executed on the server. If the server first executes put and then delete, and if the put response arrives after the delete, then `Put()` sees that the key was deleted from the cache and so stores the put's value in `lkv.leases`. This means that the cache can be updated with a stale value even after the key is deleted from the server. We discovered this bug when we were unable to prove that the LeasingInv invariant holds after `Put()` updates `lkv.leases`. Given that the `etcd` developers had moved on to a new client caching library by this point (§4.3), we reported the bug and stopped our verification attempt of this library.

Summary. Formal verification helped us find subtle issues that occur only in the presence of concurrency, multiple clients, network delays, and message re-ordering. Furthermore, we found these concurrency-related bugs without having to verify or even specify the entire leasing library; it was sufficient to attempt verifying individual functions such as `Get` and `Put`. Even though `Get` alone is not enough to manifest the lease-expiration-check bug above, we caught this bug during the proof of `Get` because it relied on the LeasingInv invariant that implicitly captured the expected behavior of other operations.

This case study again highlights the importance of code-level verification. At the protocol level, the leasing library can in principle be correct, but the specific API exposed by the session library to check for session expiration is insufficient to correctly validate a session.

Finally, the effort required in this case study is modest. The case study entailed about 1000 lines of Rocq code, which corresponds to proofs (or partial proofs) of several functions in the leasing library. For context, the leasing library itself is implemented in about 1200 lines of Go code. Moreover, we started finding issues through verification well before writing the full 1000 lines of Rocq code; as described above, we incrementally discovered problems with the leasing library as we stated invariants and proved code.

```go
func (s *store) getSnapshot(rev int64) (*snapshot, int64,
↪  error) {
  s.mu.RLock()
  defer s.mu.RUnlock()

  ...

  var targetSnapshot *snapshot
  s.history.AscendGreaterOrEqual(rev, func(rev int64, snap
↪  *snapshot) bool {
    targetSnapshot = snap
    return false
  })
  // If s.history < rev < s.latest.rev serve latest.
  if targetSnapshot == nil {
    targetSnapshot = &s.latest
  }

  return targetSnapshot, s.latest.rev, nil
}
```

Figure 12: Part of the original, buggy code from `etcd`'s `cache` library.

4.3 etcd's new cache library

As a final case study in applying formal verification, we picked `etcd`'s new `cache` library, which is under active development by `etcd` developers. First, we identified a buggy function without formal verification. Then, we implemented a fix and formally verified the new function. Along the way, verification helped us realize that our initial fix was actually still buggy. Our final, verified fix was adopted by `etcd`.

The new `cache` library is based on a cache previously implemented in Kubernetes [10] and works by monitoring the etcd servers for updates to cached keys. Since `etcd` is a multi-versioned key-value store (with each version of the key-value map being at a different "revision" number), the cache stores a history of snapshots of key-value state. When a client wants to read a specific revision of the key-value mapping, the cache searches for a snapshot at that revision in its history.

In trying to pick a starting function to verify, we did not find any existing bug reports related to `cache`. Instead, we prompted an AI model to suggest functions that are tricky (and might be buggy). This led us to the `getSnapshot` function. `getSnapshot` helps handle read requests by providing the requested revision of the cached key-value state as shown in Figure 12.

Specification. We specified `getSnapshot` as shown in Figure 13. The postcondition $\mathsf{IsSnapshot}(\texttt{snap}, m)$ denotes ownership that permits read-only access to the returned `*snapshot` pointer, and that the contents correspond to the key-value mapping m. The postcondition $\texttt{rev} \overset{\text{etcd}}{\mapsto}_{\square} m$ denotes that the requested revision corresponds to m in the etcd server's multi-versioned state.

The bugs. Even before writing any proof, we noticed two bugs in the code which violated these two postconditions. First, the code returns `&s.latest`—a pointer to an internal field of the `s` struct. This is not safe for read-only access because other threads may acquire a write lock on `s.mu` and change `s.latest`. Second, if the snapshot history does not contain the exact requested revision `rev`, the call to

$$\{\dots\}$$
`store.getSnapshot(rev)`
$$\left\{\begin{array}{l}\textbf{ret}\ (\texttt{snap},\ \texttt{r},\ \texttt{err}), \texttt{err} = \texttt{nil} \rightarrow \\ \quad \exists\, m,\ \mathsf{IsSnapshot}(\texttt{snap}, m) * \texttt{rev} \overset{\text{etcd}}{\mapsto}_{\square} m\end{array}\right\}$$

Figure 13: Spec for `getSnapshot` from `etcd`'s `cache` library.

`AscendGreaterOrEqual` can return a strictly newer revision, with values that do not correspond to revision `rev`.

For both bugs, we were able to write failing test cases. The first is a new test case with a data race (flagged by the Go race detector). The second involves a careful sequence of `Put` and `Get` operations that behave differently on the cache compared to the `etcd` servers.

Postulates. The code relies on functions from a few other libraries. For one, snapshots store key-value data using a `BTree` library. For another, the snapshot history is stored with a `ringBuffer` library, which provides functions such as `AscendGreaterOrEqual`. We read the documentation for each of these libraries, and wrote pre/post-condition postulates. For instance, the specs for the `BTree` library were reminiscent of the fractional ownership spec for Go maps from §3.2.

Trying to verify an incorrect fix. We initially attempted to fix the return of `&s.latest` by cloning the `latest` snapshot before returning. While trying to verify the spec for this version of the code, we got stuck when trying to apply our postulated spec for `BTree.Clone`. Based on `BTree`'s documentation stating that "`Clone` should not be called concurrently", the postulate for `Clone` requires exclusive ownership of the `BTree` being cloned as precondition [39]. The proof of `getSnapshot` gets ownership of `s.latest` from the postcondition of `RLock`. However, the postcondition of `RLock` only provides read-only, fractional ownership. This was insufficient to proceed in the proof, and helped us realize that the `s.latest` map *cannot* be safely cloned in `getSnapshot`.

Verifying a correct fix. To return the correct revision, `getSnapshot` should use the existing function `DescendLessOrEqual` in place of `AscendGreaterOrEqual`. For instance, if the cache has received snapshots from etcd at revision 10 and 20, then it is guaranteed that none of the cached keys changed in any intermediate revision. So, `getSnapshot(15)` should return the key-value mapping from revision 10. This is handled by `DescendLessOrEqual` looking for the largest snapshot that is at most `rev`.

Once we made this change, we further realized that `&s.latest` never needs to be returned. In the old, buggy code, `&s.latest` was returned when no relevant snapshot was found in `history`. With `DescendLessOrEqual`, the only possibilities are that either a snapshot is found in the `history` buffer or that every snapshot in the history is newer than the

requested version. The implementation already handled the latter case by returning an error `ErrCompacted`.

Summary. This case study shows how formal verification can help validate and fix new code. We verified just a single function (chosen with the help of AI for its importance and subtlety) and ruled out a data race and a functional correctness bug in the current version of the `cache` library. Our proof covered 51 lines of code with 290 lines of proof.

This case study also highlights the benefits of CSL reasoning at the code level. We realized our initial fix with `BTree.Clone` was buggy when we got stuck in a proof that did not have enough ownership to satisfy the precondition of `Clone`. This ruled out a concurrency bug without having to verify the implementation of any other functions (such as the ones that mutate the `BTree`).

5 Related work

One important aspect of our case studies is that they apply to the code of a system. This is in contrast to prior work on protocol verification, such as the use of TLA+ at AWS [29]. As the AWS developers observe, protocol-level verification helps catch design issues in the protocol itself. However, it is not in a position to help catch similar design issues in the implementation's concurrency plan, nor bugs in the code of the implementation. Moreover, it requires fully defining the protocol specification; in our case study with the read-only optimization and with the leasing cache, we did not have to specify the complete protocol.

Prior work on formal verification of system implementations has focused on verifying clean-slate code written with verification in mind, typically as part of the verification project [6, 11, 13, 15, 23, 25, 26, 36, 40, 42]. Many of these projects are, of course, motivated by real-world systems, but tend to encapsulate the core verification challenges in a clean-slate implementation, such as the work on verifying Kubernetes controllers [38]. A key difference in this paper is the focus on applying formal verification to parts of real-world systems, which were not written for verification.

Verifying off-the-shelf code requires a verification tool that can process all of the language features used in real-world code bases, as Dingo is able to do for Go code. The Spoq tool [27] is similarly able to process large amounts of unmodified C code. To verify its generated Rocq code, Spoq uses the CCAL approach [13], which requires organizing functions into layers, and proving complete refinements between low-level implementation layers and higher-level specifications. In contrast, the CSL approach allows for verifying a single function at a time while still getting value. CN [34] is a separation logic-based verification tool that similarly targets a large fragment of unmodified C code, and has been used to verify the buddy allocator from the pKVM hypervisor. However, unlike Dingo, CN does not model concurrency, and thus would have trouble applying to distributed systems.

Gobra [41] is a verification tool for Go that has been used to verify an off-the-shelf router for the SCION protocol [33]. Gobra is based on implicit dynamic frames, which supports a form of modular verification similar to CSL [32]. Gobra verification of the SCION router proves a complete refinement between the code and the protocol, as opposed to the CSL approach in §2 that specifies concurrency and distributed execution using ghost callbacks. The main contribution of this paper is in showing that CSL provides substantial value from verifying just a few functions, chosen by the developer, and that it might not be necessary to verify the full system for verification to be useful.

All the verification techniques used in this paper are well known, and have been used for formal verification of complete systems. Indeed, Dingo builds on the Grove framework, which used the same CSL-based modular verification machinery to verify a complete clean-slate implementation of a distributed key-value store. The contribution of this work lies in the observation that these same techniques can be profitably used for verifying existing distributed systems not written with verification in mind, and that it's valuable to verify even small parts of a system.

Lightweight verification and bug-finding. Lightweight verification and bug-finding techniques have been effective at helping developers find bugs in real-world systems [1, 3, 5] and checking for specific properties of interest, such as isolation in TickTock [35] and VeriWasm [20]. Indeed, a key design goal for these tools is to target real-world systems. In that light, this paper's contribution can be thought of as showing how "heavyweight" formal verification techniques can be applicable to real-world systems as well. One benefit of doing so is that these "heavyweight" verification techniques can help developers reason about arbitrary safety properties, such as linearizability and consistency in a distributed system, and the verification approach does not a priori restrict the kinds of properties that can be checked.

6 Conclusion

This paper showed that Concurrent Separation Logic can be profitably applied to existing distributed systems to find and fix serious issues, as demonstrated by our three `etcd` case studies using our new Dingo tool. We got substantial value by verifying just a few functions at a time, including being able to make a strong case for `etcd` developers to accept our more aggressive fix for the read-only optimization. The key was CSL's ability to encode global properties like linearizability in local per-function specifications and our approach of writing invariants and postulates for other parts of the codebase. We hope that our results are compelling and convince other developers to embrace formal verification and CSL.

References

[1] AdaCore. SPARK Pro. `https://www.adacore.com/sparkpro`, 2025.

[2] Anish Athalye, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying hardware security modules with information-preserving refinement. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 503–519, Carlsbad, CA, July 2022.

[3] James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, and Andrew Warfield. Using lightweight formal methods to validate a key-value storage node in Amazon S3. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, Virtual conference, October 2021.

[4] Mike Burrows. The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Seattle, WA, November 2006.

[5] Cristiano Calcagno, Dino Distefano, Jeremy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O'Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. Moving fast with software verification. In *Proceedings of the 7th Annual NASA Formal Methods Symposium (NFM)*, Pasadena, CA, April 2015.

[6] Tej Chajed, Joseph Tassarotti, Mark Theng, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying the DaisyNFS concurrent and crash-safe file system with sequential reasoning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 447–463, Carlsbad, CA, July 2022.

[7] Yun-Sheng Chang, Ralf Jung, Upamanyu Sharma, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying vMVCC, a high-performance transaction library using multi-version concurrency control. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Boston, MA, July 2023.

[8] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nickolai Zeldovich. Using Crash Hoare Logic for certifying the FSCQ file system. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP)*, pages 18–37, Monterey, CA, October 2015.

[9] Xiangdong Chen, Zhaofeng Li, Jerry Zhang, Vikram Narayanan, and Anton Burtsev. Atmosphere: Practical verified kernels with Rust and Verus. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, Seoul, South Korea, October 2025.

[10] Etcd Developers. Develop a caching library for etcd, 2025. URL `https://github.com/etcd-io/etcd/issues/19371`.

[11] Andrew Ferraiuolo, Andrew Baumann, Chris Hawblitzel, and Bryan Parno. Komodo: Using verification to disentangle secure-enclave hardware from software. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*, pages 287–305, Shanghai, China, October 2017.

[12] Léon Gondelman, Simon Oddershede Gregersen, Abel Nieto, Amin Timany, and Lars Birkedal. Distributed causal memory: modular specification and verification in higher-order distributed separation logic. In *Proceedings of the 48th ACM Symposium on Principles of Programming Languages (POPL)*, Virtual conference, January 2021.

[13] Ronghui Gu, Zhong Shao, Jieung Kim, Xiongnan Wu, Jérémie Koenig, Vilhelm Sjöberg, Hao Chen, David Costanzo, and Tahina Ramananandro. Certified concurrent abstraction layers. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 646–661, Philadelphia, PA, June 2018.

[14] Travis Hance, Andrea Lattuada, Chris Hawblitzel, Jon Howell, Rob Johnson, and Bryan Parno. Storage systems are distributed systems (so verify them that way!). In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 99–115, Virtual conference, November 2020.

[15] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. IronFleet: Proving practical distributed systems correct. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP)*, pages 1–17, Monterey, CA, October 2015.

[16] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: a correctness condition for concurrent objects. *ACM Transactions on Programming Languages Systems*, 12 (3):463–492, 1990.

[17] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, October 1969.

[18] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. Zookeeper: wait-free coordination for internet-scale systems. In *Proceedings of the*

2010 USENIX Conference on USENIX Annual Technical Conference, USENIXATC'10, page 11, USA, 2010. USENIX Association.

[19] Bart Jacobs and Frank Piessens. Expressive modular fine-grained concurrency specification. In *Proceedings of the 38th ACM Symposium on Principles of Programming Languages (POPL)*, pages 271–282, Austin, TX, January 2011.

[20] Evan Johnson, David Thien, Yousef Alhessi, Shravan Narayan, Fraser Brown, Sorin Lerner, Tyler McMullen, Stefan Savage, and Deian Stefan. Доверяй, но проверяй: SFI safety for native-compiled Wasm. In *Proceedings of the 2021 Annual Network and Distributed System Security Symposium (NDSS)*, Virtual conference, February 2021.

[21] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: a modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.

[22] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Michael Norrish, Rafal Kolanski, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, Big Sky, MT, October 2009.

[23] Gerwin Klein, June Andronick, Kevin Elphinstone, Toby Murray, Thomas Sewell, Rafal Kolanski, and Gernot Heiser. Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems*, 32(1):2:1–70, February 2014.

[24] Hayley LeBlanc, Jacob R. Lorch, Chris Hawblitzel, Cheng Huang, Yiheng Tao, Nickolai Zeldovich, and Vijay Chidambaram. PoWER never corrupts: Tool-agnostic verification of crash consistency and corruption detection. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Boston, MA, July 2025.

[25] Mohsen Lesani, Christian J. Bell, and Adam Chlipala. Chapar: Certified causally consistent distributed key-value stores. In *Proceedings of the 43rd ACM Symposium on Principles of Programming Languages (POPL)*, pages 357–370, St. Petersburg, FL, January 2016.

[26] Xupeng Li, Xuheng Li, Christoffer Dall, Ronghui Gu, Jason Nieh, Yousuf Sait, and Gareth Stockwell. Design and verification of the Arm confidential compute architecture. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Carlsbad, CA, July 2022.

[27] Xupeng Li, Xuheng Li, Wei Qiang, Ronghui Gu, and Jason Nieh. Spoq: Scaling machine-checkable systems verification in Coq. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Boston, MA, July 2023.

[28] Luke Nelson, Jacob Van Geffen, Emina Torlak, and Xi Wang. Specification and verification in the field: Applying formal methods to BPF just-in-time compilers in the Linux kernel. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 41–61, Virtual conference, November 2020.

[29] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon Web Services uses formal methods. *Communications of the ACM*, 58(4):66–73, April 2015.

[30] Peter W. O'Hearn. Resources, concurrency, and local reasoning. *Theoretical Computer Science*, 375(1):271–307, 2007.

[31] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference*, pages 305–319, Philadelphia, PA, June 2014.

[32] Matthew J. Parkinson and Alexander J. Summers. The relationship between separation logic and implicit dynamic frames. *Log. Methods Comput. Sci.*, 8(3), 2012. doi: 10.2168/LMCS-8(3:1)2012. URL `https://doi.org/10.2168/LMCS-8(3:1)2012`.

[33] João C. Pereira, Tobias Klenze, Sofia Giampietro, Markus Limbeck, Dionysios Spiliopoulos, Felix Wolf, Marco Eilers, Christoph Sprenger, David A. Basin, Peter Müller, and Adrian Perrig. Protocols to code: Formal verification of a secure next-generation internet router. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, pages 1469–1483. ACM, 2025.

[34] Christopher Pulte, Dhruv C. Makwana, Thomas Sewell, Kayvan Memarian, Peter Sewell, and Neel Krishnaswami. CN: verifying systems C code with separation-logic refinement types. *Proc. ACM Program. Lang.*, 7 (POPL):1–32, 2023.

[35] Vivien Rindisbacher, Evan Johnson, Nico Lehmann, Tyler Potyondy, Pat Pannuto, Stefan Savage, Deian Stefan, and Ranjit Jhala. TickTock: Verified isolation in a production embedded OS. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, Seoul, South Korea, October 2025.

[36] Upamanyu Sharma, Ralf Jung, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. Grove: a separation-logic library for verifying distributed systems. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, Koblenz, Germany, October 2023.

[37] Naomi Smith, Abhishek Sharma, John Renner, David Thien, Fraser Brown, Hovav Shacham, Ranjit Jhala, and Deian Stefan. Icarus: Trustworthy just-in-time compilers with symbolic meta-execution. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*, Austin, TX, November 2024.

[38] Xudong Sun, Wenjie Ma, Jiawei Tyler Gu, Zicheng Ma, Tej Chajed, Jon Howell, Andrea Lattuada, Oded Padon, Lalith Suresh, Adriana Szekeres, and Tianyin Xu. Anvil: Verifying liveness of cluster management controllers. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Santa Clara, CA, July 2024.

[39] The Kubernetes Authors. btree package - k8s.io/utils/third_party/forked/golang/btree - Go Packages, 2026. URL `https://pkg.go.dev/k8s.io/utils/third_party/forked/golang/btree#BTree.Clone`.

[40] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. Verdi: A framework for implementing and formally verifying distributed systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 357–368, Portland, OR, June 2015.

[41] Felix A. Wolf, Linard Arquint, Martin Clochard, Wytse Oortwijn, João Carlos Pereira, and Peter Müller. Gobra: Modular specification and verification of go programs. In *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part I*, volume 12759 of *Lecture Notes in Computer Science*, pages 367–379. Springer, 2021.

[42] Ziqiao Zhou, Anjali, Weiteng Chen, Sishuai Gong, Chris Hawblitzel, and Weidong Cui. VeriSMo: A verified security module for confidential VMs. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Santa Clara, CA, July 2024.